

# Class 5: Data Viz with ggplot

Laura (A17844089)

Today we are exploring the **ggplot** package and how to make nice figures in R.

There are lots of ways to make figures and plot in R. These include:

- so called “base” R
- and add on package like **ggplot2**

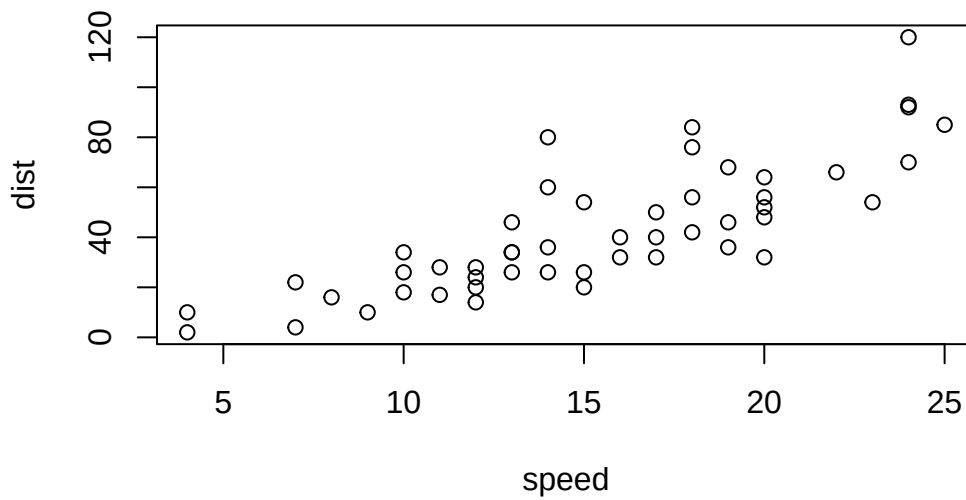
Here is a simple “base” R plot.

```
head(cars)
```

|   | speed | dist |
|---|-------|------|
| 1 | 4     | 2    |
| 2 | 4     | 10   |
| 3 | 7     | 4    |
| 4 | 7     | 22   |
| 5 | 8     | 16   |
| 6 | 9     | 10   |

We can simply pass to to the `plot()` function.

```
plot(cars)
```



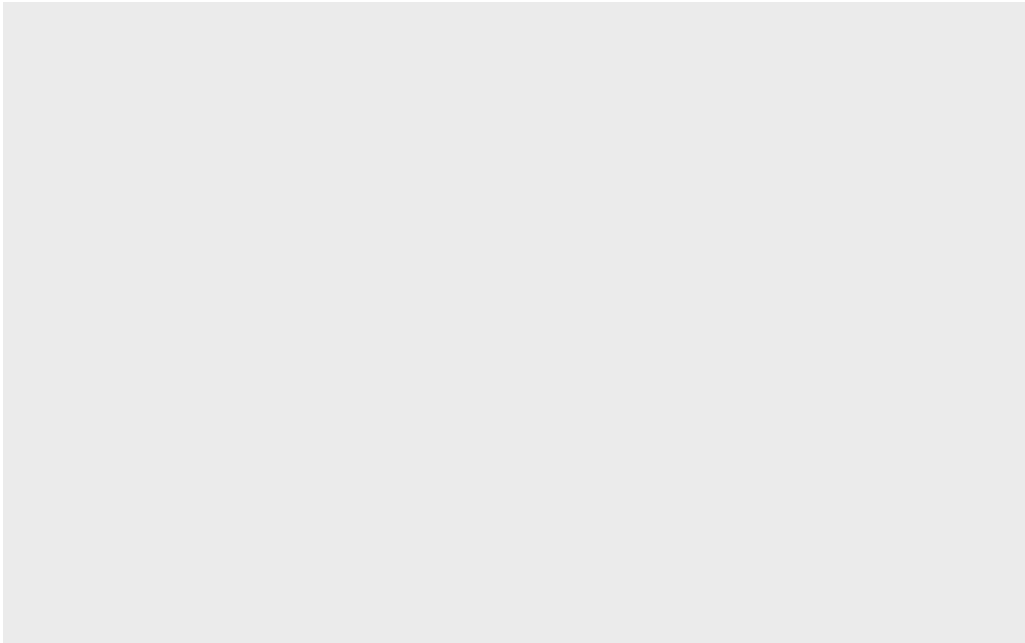
Key-point: Base R is quick but not so nice and simple looking in some folks eyes

Let's see how we can plot this with **ggplot2**...

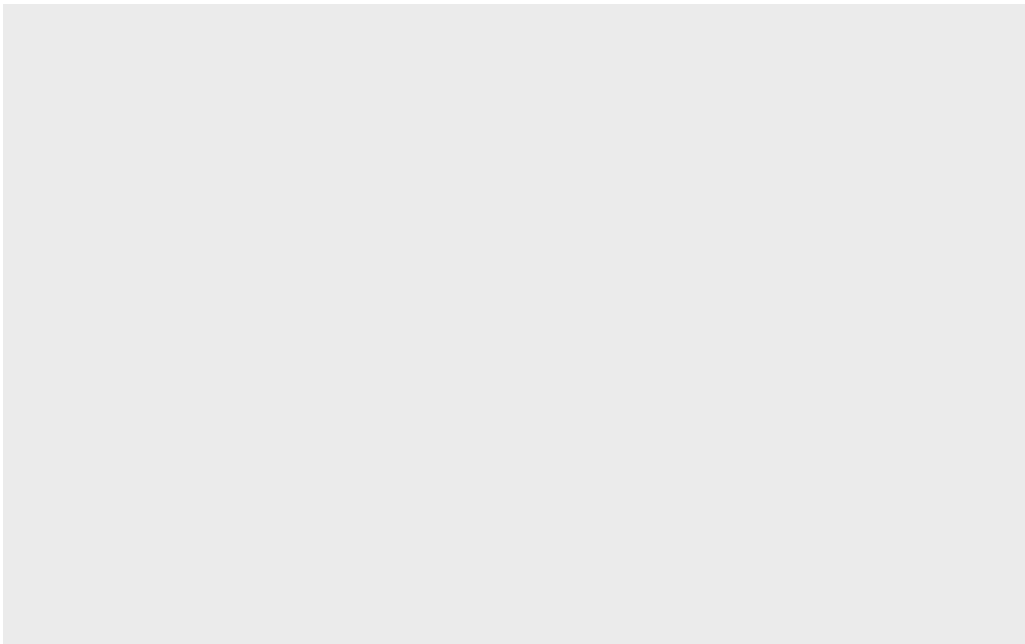
1st I need to install this add-on package. For this we use the `install.packages()` function - **WE DO THIS IN THE CONSOLE, NOT our report**. This is a one time only deal.

2nd We need to load the package with the `library()` function every time we want to use it.

```
library(ggplot2)
ggplot(cars)
```



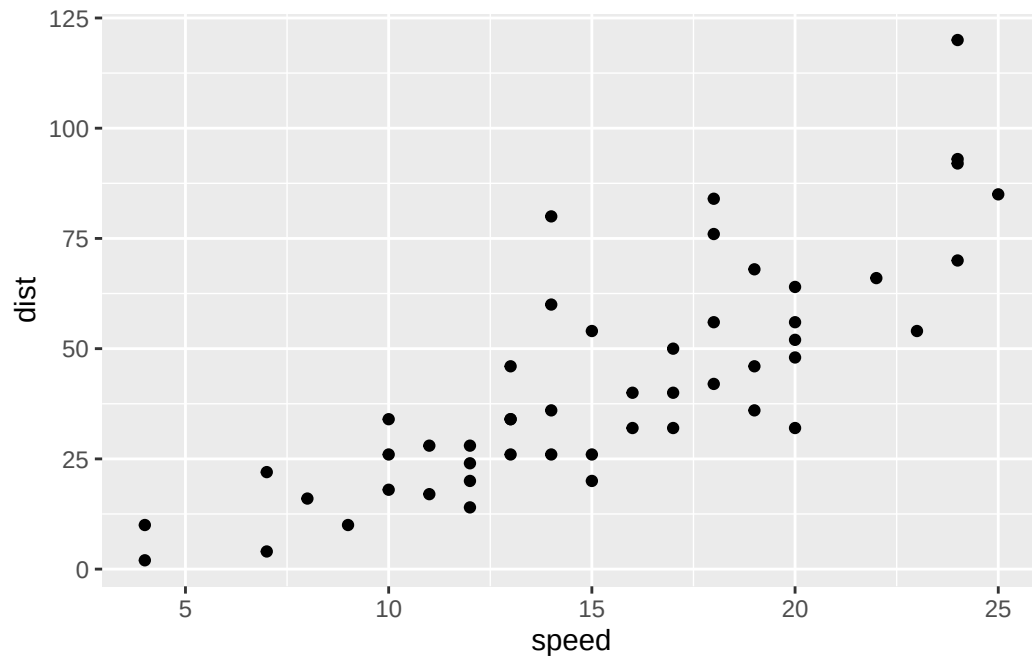
```
ggplot(cars)
```



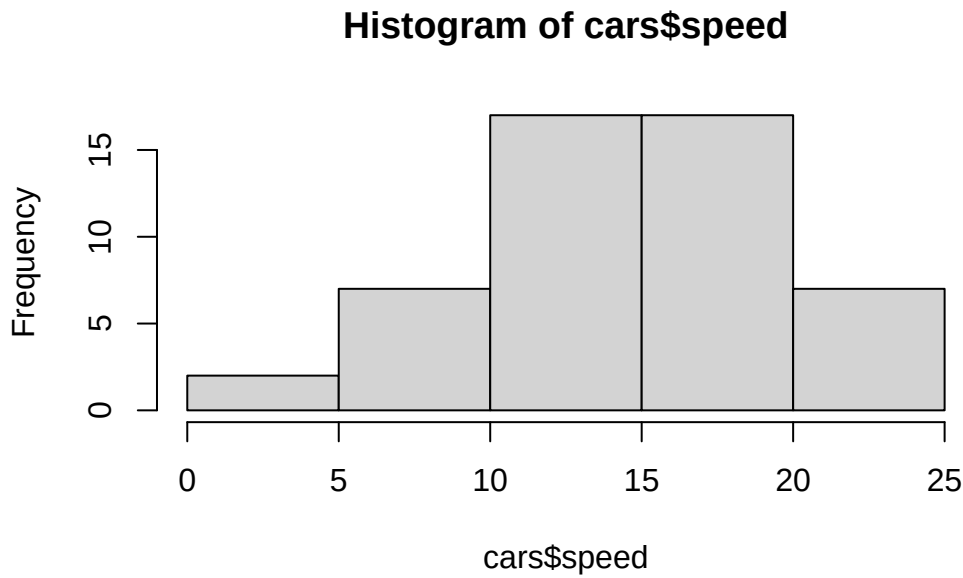
Every ggplot is composed of at least 3 layers:

- **data** (i.e a data.frame with the things you want to plot),
- aesthetics **aes()** that map the columns of data to your plot features (i.e aesthetics)
- geoms like **geom\_point()** that srt how the plot appears

```
library(ggplot2)
ggplot(cars) +
  aes(x=speed, y=dist) +
  geom_point()
```



```
hist(cars$speed)
```



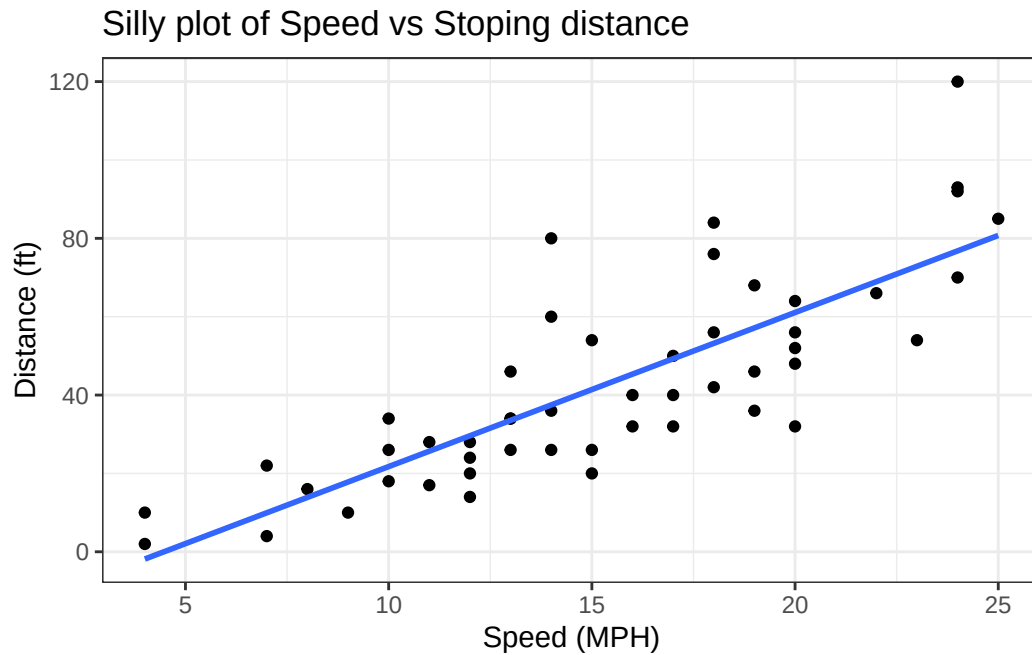
Key point: For simple “canned” graphs base R is quicker but as things get more custom and elaborate then ggplot wins out...

Let's add more layers to our ggplot

Add a line showing the relationship between x and y Add a title Add custom axis labels “Speed (MPH)” and “Distance (ft)” Change the theme...

```
ggplot(cars) +  
  aes(x=speed, y=dist) +  
  geom_point() +  
  geom_smooth(method="lm", se=FALSE) +  
  labs(title = "Silly plot of Speed vs Stopping distance",  
        x="Speed (MPH)",  
        y="Distance (ft)") +  
  theme_bw()
```

`geom\_smooth()` using formula = 'y ~ x'



## Going further

Read some gene expression data

```
url <- "https://bioboot.github.io/bimm143_S20/class-material/up_down_expression.txt"
genes <- read.delim(url)
head(genes)
```

|   | Gene       | Condition1 | Condition2 | State      |
|---|------------|------------|------------|------------|
| 1 | A4GNT      | -3.6808610 | -3.4401355 | unchanging |
| 2 | AAAS       | 4.5479580  | 4.3864126  | unchanging |
| 3 | AASDH      | 3.7190695  | 3.4787276  | unchanging |
| 4 | AATF       | 5.0784720  | 5.0151916  | unchanging |
| 5 | AATK       | 0.4711421  | 0.5598642  | unchanging |
| 6 | AB015752.4 | -3.6808610 | -3.5921390 | unchanging |

Q1. How many genes are in this wee dataset?

```
nrow(genes)
```

```
[1] 5196
```

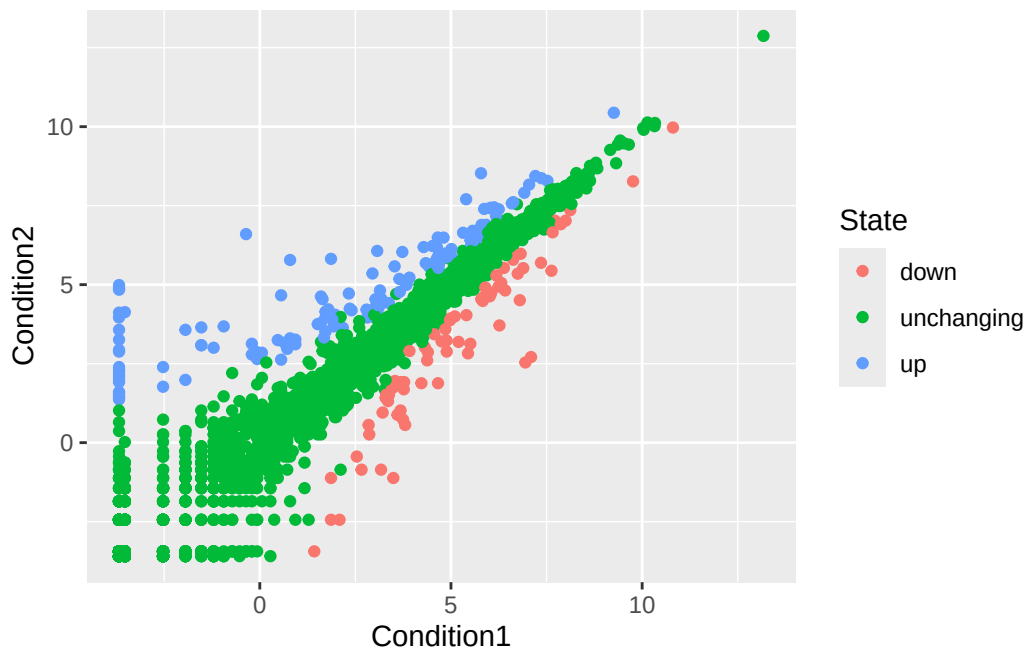
Q2. How many “up” regulated genes are there?

```
sum( genes$State == "up" )
```

```
[1] 127
```

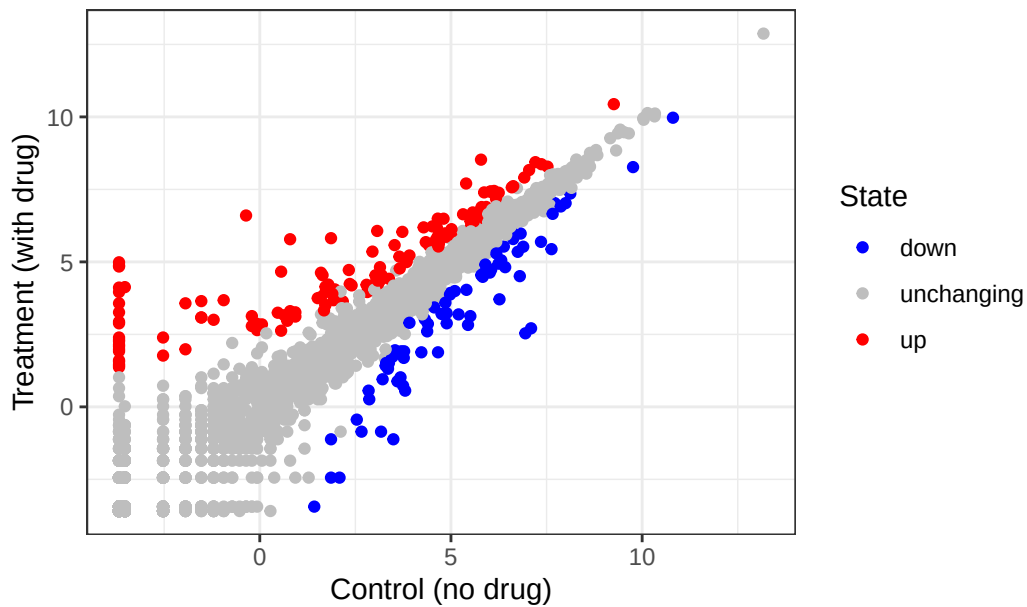
A useful function for counting up occurrences of things in a vector is the `table()` function. Make a v1 figure

```
p <- ggplot(genes) +  
  aes(x=Condition1,  
      y=Condition2,  
      col=State) +  
  geom_point()  
p
```



```
p +  
  scale_colour_manual( values= c("blue","gray","red") ) +  
  labs(title= "Expression changes upon drug treatment",  
       x="Control (no drug)",  
       y= "Treatment (with drug)" ) +  
  theme_bw()
```

Expression changes upon drug treatment



```
# File location online
url <- "https://raw.githubusercontent.com/jennybc/gapminder/master/inst/extdata/gapminder.tsv"

gapminder <- read.delim(url)
```

Lets have a wee peak

```
head( gapminder, 3)
```

|   | country     | continent | year | lifeExp | pop      | gdpPercap |
|---|-------------|-----------|------|---------|----------|-----------|
| 1 | Afghanistan | Asia      | 1952 | 28.801  | 8425333  | 779.4453  |
| 2 | Afghanistan | Asia      | 1957 | 30.332  | 9240934  | 820.8530  |
| 3 | Afghanistan | Asia      | 1962 | 31.997  | 10267083 | 853.1007  |

Q4. How many different country values are in this dataset?

```
head( gapminder, 3)
```

|   | country     | continent | year | lifeExp | pop      | gdpPercap |
|---|-------------|-----------|------|---------|----------|-----------|
| 1 | Afghanistan | Asia      | 1952 | 28.801  | 8425333  | 779.4453  |
| 2 | Afghanistan | Asia      | 1957 | 30.332  | 9240934  | 820.8530  |
| 3 | Afghanistan | Asia      | 1962 | 31.997  | 10267083 | 853.1007  |



```
length( table(gapminder$country) )
```

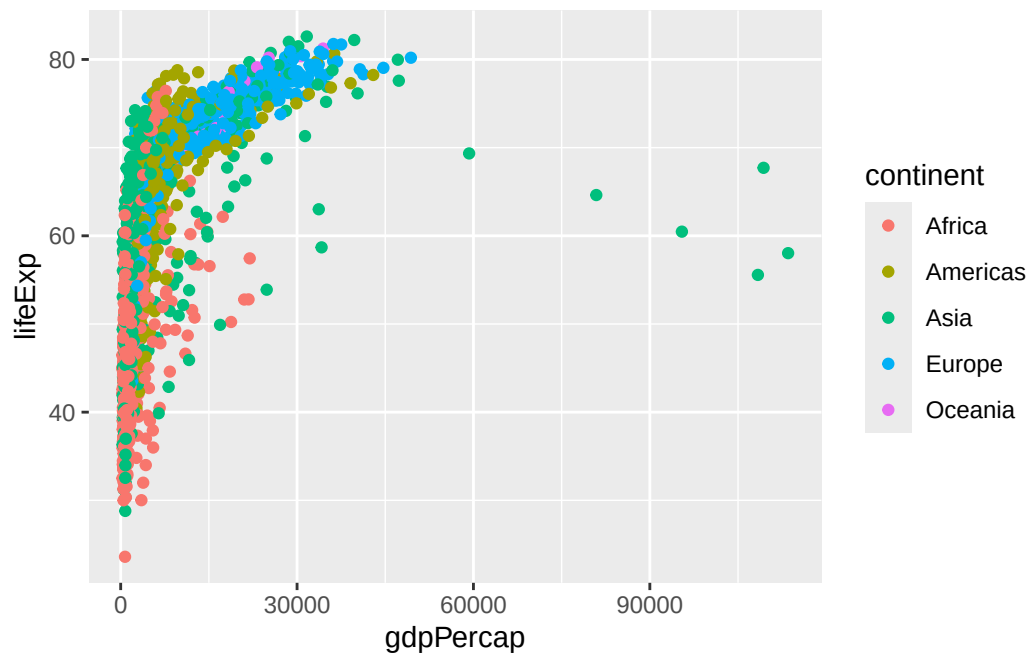
```
[1] 142
```

Q5. How many different continent values are in the dataset?

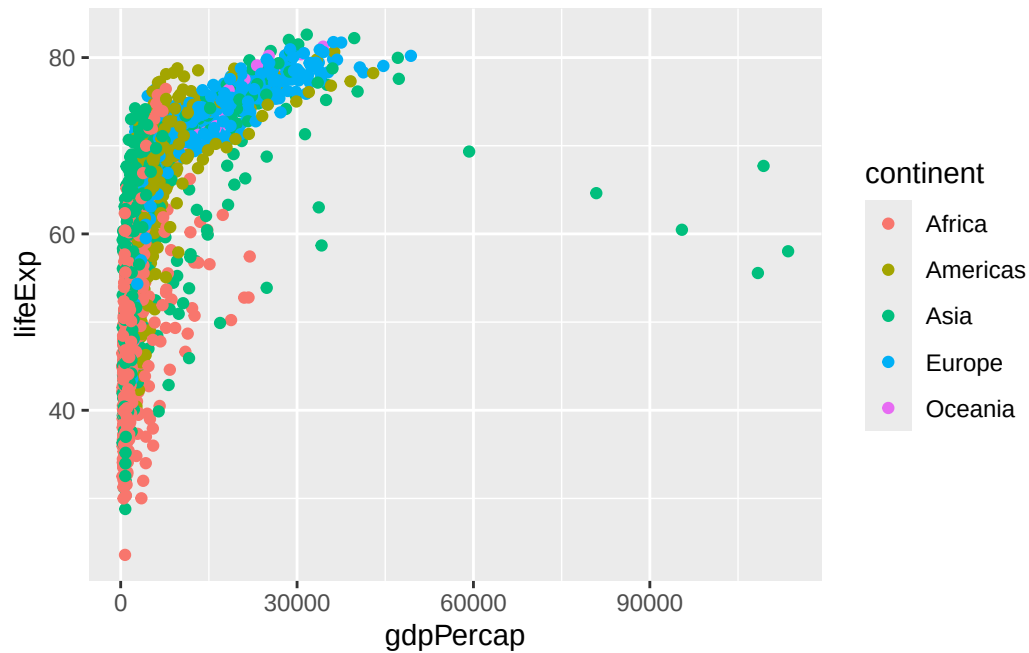
```
unique(gapminder$continent)
```

NULL

```
ggplot(gapminder) +  
  aes(gdpPercap, lifeExp, col=continent) +  
  geom_point()
```



```
ggplot(gapminder) +  
  aes(gdpPercap, lifeExp, col=continent, label=country) + geom_point()
```



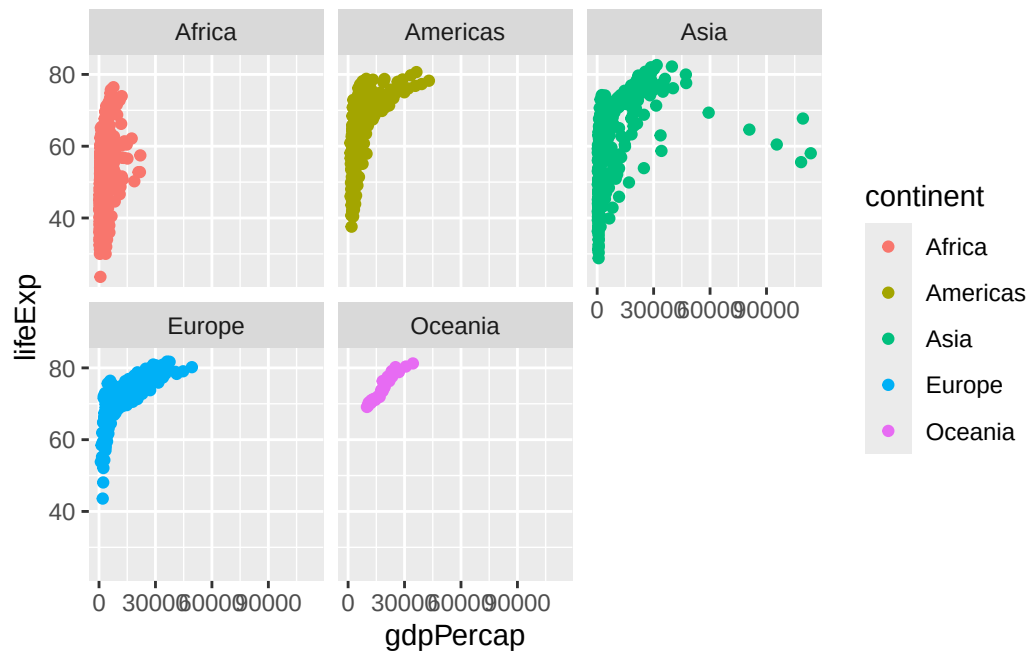
```
geom_point()
```

```
geom_point: na.rm = FALSE
stat_identity: na.rm = FALSE
position_identity
```

I can use the **ggrep** package to make more sensible labels here.

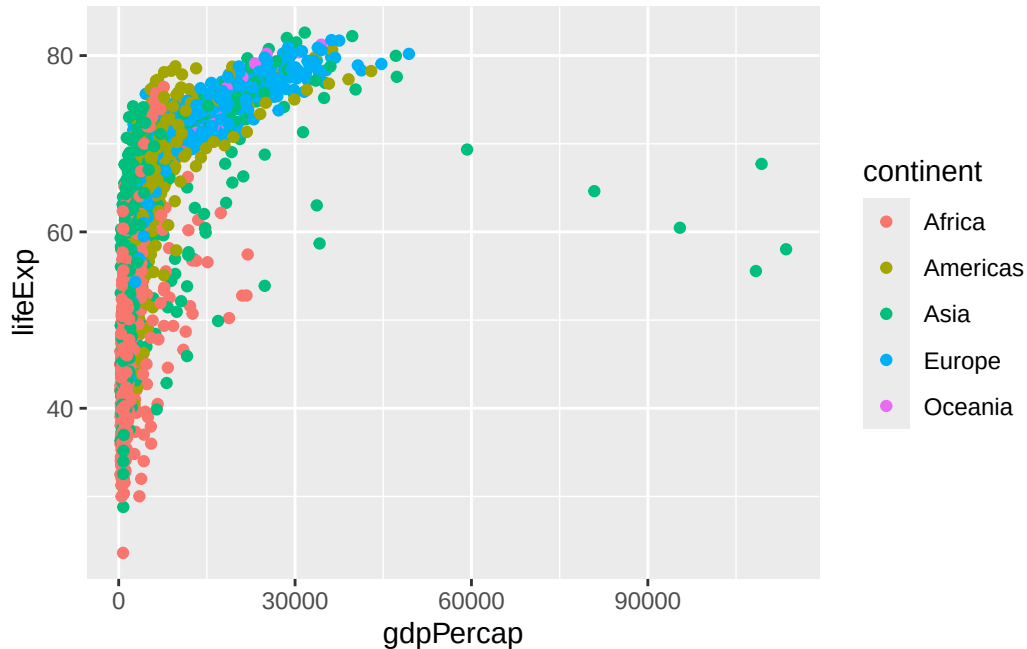
I want a separate panel per continent

```
ggplot(gapminder) +
  aes(gdpPerCap, lifeExp, col=continent, label=country) + geom_point() +
  facet_wrap(~continent)
```



```
library(ggplot2)

ggplot(gapminder) +
  aes(gdpPercap, lifeExp, col=continent, label=country) + geom_point()
```



The advantages of ggplot over base R plot: Layered system: ggplot builds plots by adding layers (data, aesthetics, geometry, themes, annotations) with the `+` operator, making it easy to iteratively refine and customize visualizations. Base R requires step-by-step specification, which can be more fiddly and time-consuming to polish D1 D2 D4 D5 D6 Q2 . Aesthetic mapping: ggplot allows mapping data columns to visual features like position, color, shape, size, and transparency, supporting rich, multi-dimensional plots. Base R is less flexible in this regard D1 D2 D3 D5 D6 Q1 Q3 . Themes and annotation: ggplot provides built-in themes and functions for adding titles, subtitles, captions, and custom axis labels, making it easier to standardize and polish figures D1 D2 D4 D5 D6 Q1 . Faceting: ggplot can split data into multiple panels by categories using `facet_wrap`, improving clarity for complex or grouped data D1 D2 D4 D5 D6 Q1 . Variety of plot types: ggplot supports over 40 core geometric types and many more via extensions, making advanced visualizations more accessible D1 D2 D6 Q3 . In summary, ggplot is more structured, modular, and user-friendly for creating publication-quality figures, while base R offers complete control but is less convenient for complex or polished visualizations D1 D2 D4 D5 D6 Q1 Q2 Q3 .